

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут телекомунікацій, радіоелектроніки та електронної техніки
кафедра «Радіоелектронні пристрої та системи»



Звіт з лабораторної роботи №2
з дисципліни «Вимірювання в радіоелектроніці »

Підготував:
ст. групи ІХ-21
Щурко Максим Олександрович

Прийняла:
Гордійчук-Бублівська О.В

Львів 2025р.

Тема роботи: Моделювання телекомунікаційної мережі з використанням генераторів подій, протоколів TCP/UDP, асинхронного моделювання. Розробка графічної візуалізації мережі та аналіз продуктивності.

Мета роботи: Ознайомлення з основами моделювання телекомунікаційних мереж. Вивчення використання об'єктно-орієнтованого програмування для моделювання мережі. Моделювання різних топологій мереж (зірка, кільцева, деревоподібна, сіткова (mesh), комбінована тощо). Симуляція передачі пакетів між вузлами та маршрутизаторами з використанням TCP та UDP. Вимірювання продуктивності мережі: швидкість передачі, статистика втрат, затримки передачі. Побудова візуалізації для графічного відображення структури та роботи мережі.

Теоретичні відомості

Об'єктно-орієнтоване програмування в Python

Python підтримує ООП, що дозволяє створювати класи для моделювання реальних об'єктів, таких як вузли, маршрутизатори, пакети та їх взаємодію.

2. Мережеві протоколи: TCP і UDP

TCP (Transmission Control Protocol) — це протокол транспортного рівня в моделі OSI, який забезпечує надійну та гарантовану передачу даних між двома комп'ютерами або пристроями через мережу. Він забезпечує:

Гарантовану доставку: Пакети даних доставляються в порядку їх відправлення, і якщо якийсь пакет не досягає одержувача, протокол спробує передати його знову.

Підтвердження доставки: Отримувач посилає підтвердження для кожного отриманого пакету, що дає відправнику знати, що пакет успішно доставлений.

- Контроль помилок: TCP використовує контрольні суми для перевірки наявності помилок у переданих даних. Якщо помилка виявлена, дані переправляються знову.

- Управління потоком і контроль перевантаження: Протокол регулює швидкість передачі даних, щоб уникнути перевантаження мережі та пристроїв.
- З'єднання: TCP є з'єднувальним протоколом, що означає, що передача даних починається тільки після встановлення з'єднання між відправником і отримувачем (через трьохсторонній «handshake»).
- UDP (User Datagram Protocol) – це без з'єднувальний протокол транспортного рівня, який забезпечує менш надійну передачу даних, ніж TCP. Він не гарантує доставку, порядок доставки або підтвердження отримання пакету. Його основні особливості:
- UDP не встановлює з'єднання перед передачею даних, тому пакети можуть бути відправлені швидше, але без гарантії доставки.
- Відсутність підтверджень: У протоколі UDP немає механізму підтвердження отримання пакета, що робить його швидким, але ненадійним.
- Швидкість: Завдяки відсутності підтверджень і додаткових перевірок UDP може бути швидшим, ніж TCP.
- Простота: UDP має мінімальний набір функцій, що робить його простим у реалізації, але менш стабільним для критичних ітераторів, де важлива надійність.

3. Генератори подій

Генератори подій (або event generators) в Python — це механізми, які дозволяють створювати події та обробляти їх асинхронно або по мірі їх виникнення. Це корисно для створення програм, де важливо реагувати на події, що відбуваються в реальному часі, наприклад, в інтерфейсах користувача, в іграх або в серверах.

В Python є кілька підходів для реалізації таких механізмів, зокрема, використання:

- Генераторів (Generators) — вони дозволяють створювати ітератори для обробки подій.
- Асинхронних подій за допомогою `asyncio` — в Python є модуль `asyncio`, який дозволяє працювати з асинхронними подіями.
- Патерн Observer — цей патерн використовує підписку на події та їх сповіщення.

Візуалізація мережі

Для створення графічного представлення мережі та візуалізації різних топологій можна використати бібліотеки `networkx` та `matplotlib` в Python. Ці бібліотеки дозволяють будувати, моделювати та візуалізувати мережі, такі як комп'ютерні мережі, графи або зв'язки між об'єктами.

4.1. Визначення основних концепцій

Вузли та з'єднання (Edges)

- **Вузли (nodes):** В мережі вузли можуть представляти будь-які елементи, які з'єднані один з одним, наприклад, комп'ютери, сервери або маршрутизатори.
- **З'єднання (edges):** З'єднання між вузлами в мережі можуть бути представлені як ребра, що з'єднують два вузли. Ці з'єднання можуть бути направленими (як в Інтернеті) або ненаправленими (як в локальних мережах).

Типи топологій мережі

Існує кілька стандартних топологій, які використовуються для побудови мереж. Найпоширенішими є:

- Зіркова (Star)
- Кільцева (Ring)
- Деревоподібна (Tree)
- Сіткова (Mesh)

- Гібридна (Hybrid)
- Шинна (Bus)
- Повнозв'язна

Задання

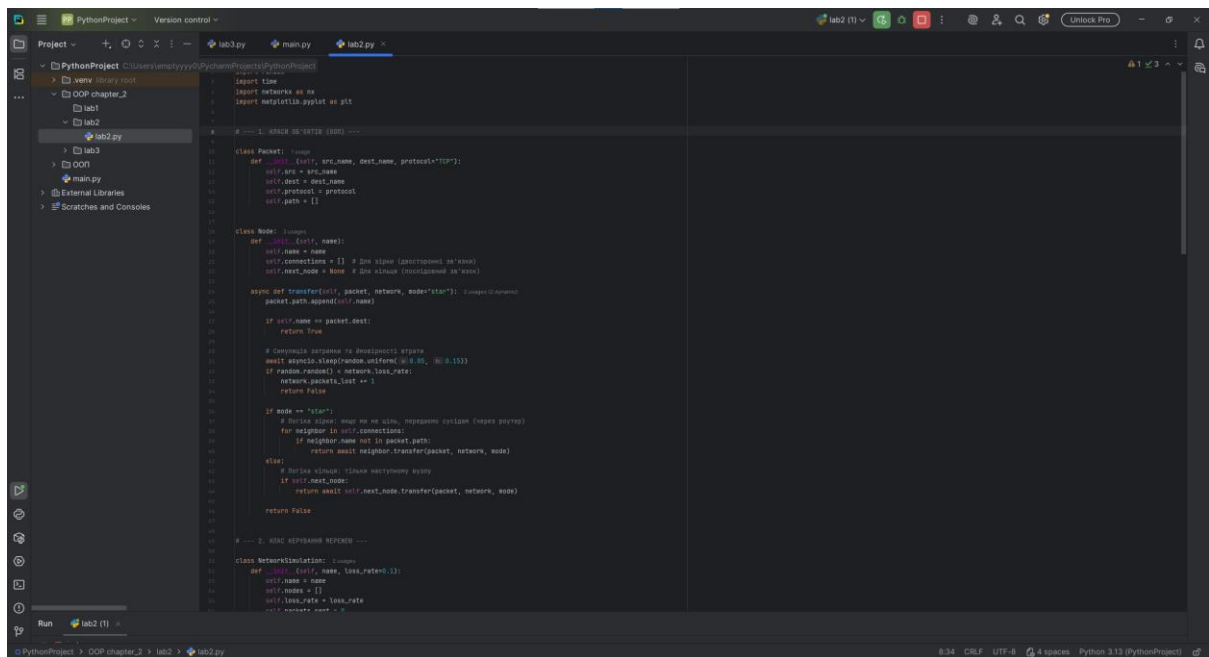


Рис1(Частина завдання)

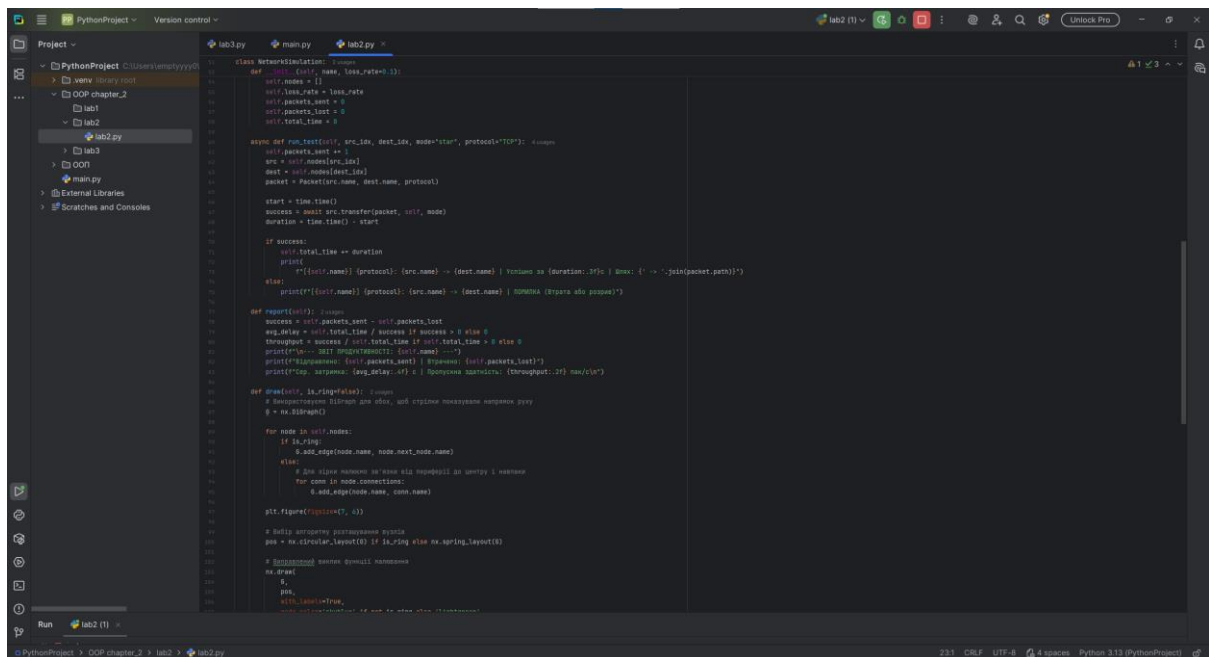


Рис.2(Друга частина завдання)

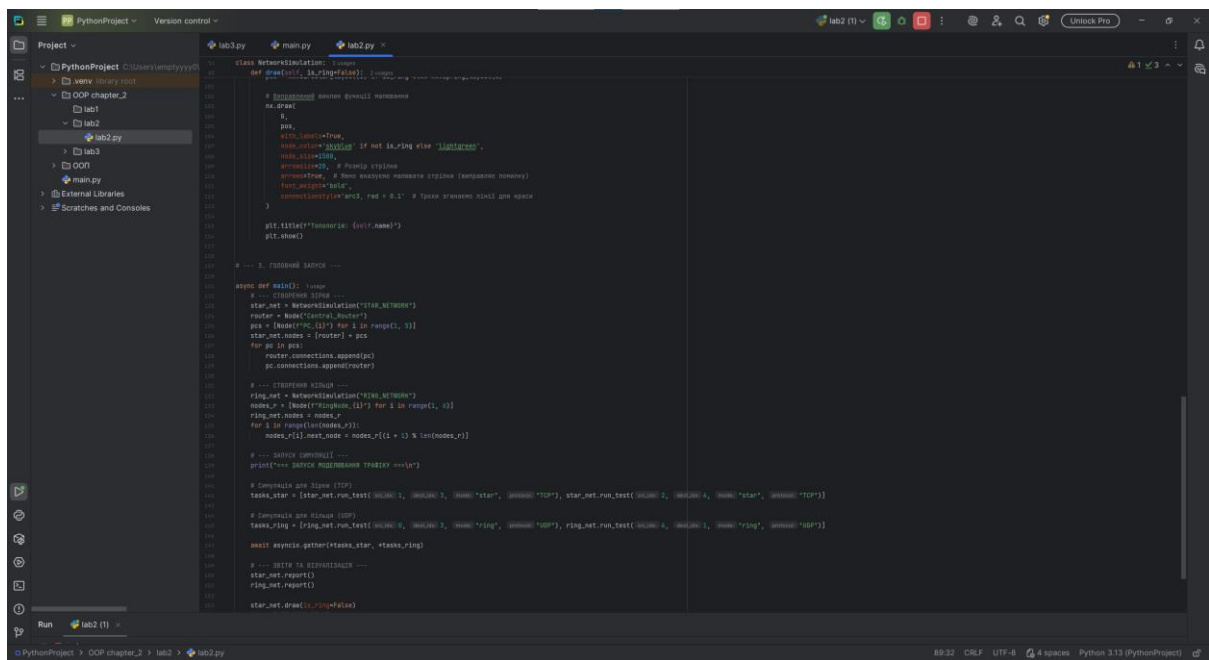


Рис.3(Третя частина завдання)

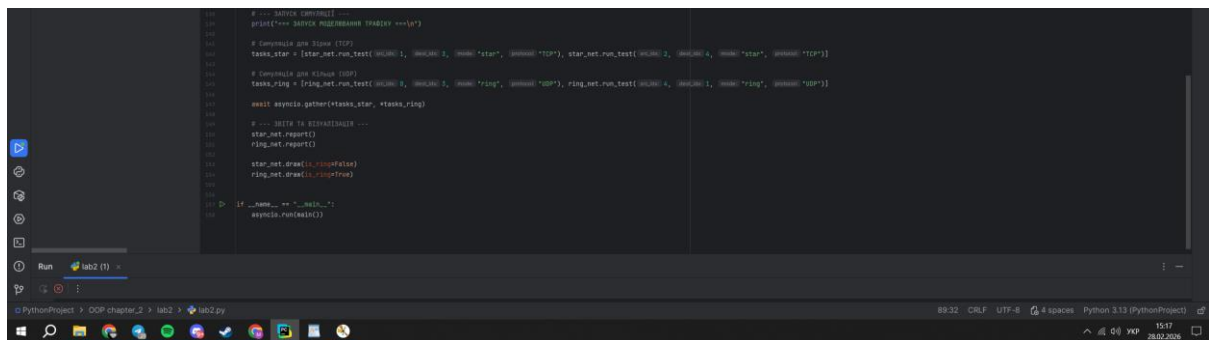


Рис.3(Фінальна частина завдання)

```
Run lab2 (1)
C:\Users\leptyyyy\PycharmProjects\PythonProject\venv\Scripts\python.exe "C:\Users\leptyyyy\PycharmProjects\PythonProject\00P_chapter_2\lab2.py"
=== ЗАДАЧА МОДЕЛЮВАННЯ ТРАДИКВ ===

[RING_NETWORK] UDP: RingNode_1 -> RingNode_4 | ПОМИЛКА (Втрата або розрив)
[RING_NETWORK] UDP: RingNode_5 -> RingNode_2 | Успішно за 0.191с | Шлях: RingNode_5 -> RingNode_1 -> RingNode_2
[STAR_NETWORK] TCP: PC_3 -> PC_3 | ПОМИЛКА (Втрата або розрив)
[STAR_NETWORK] TCP: PC_2 -> PC_4 | ПОМИЛКА (Втрата або розрив)

--- ЗБІТ ПРОДУКТИВНОСТІ: STAR_NETWORK ---
Відправлено: 2 | Втрачено: 0
Сер. затримка: 0.0000 с | Пропускна здатність: 0.00 пак/с

--- ЗБІТ ПРОДУКТИВНОСТІ: RING_NETWORK ---
Відправлено: 2 | Втрачено: 1
Сер. затримка: 0.1915 с | Пропускна здатність: 0.22 пак/с

Process finished with exit code 0
```

Рис.4(Скріншот терміналу)

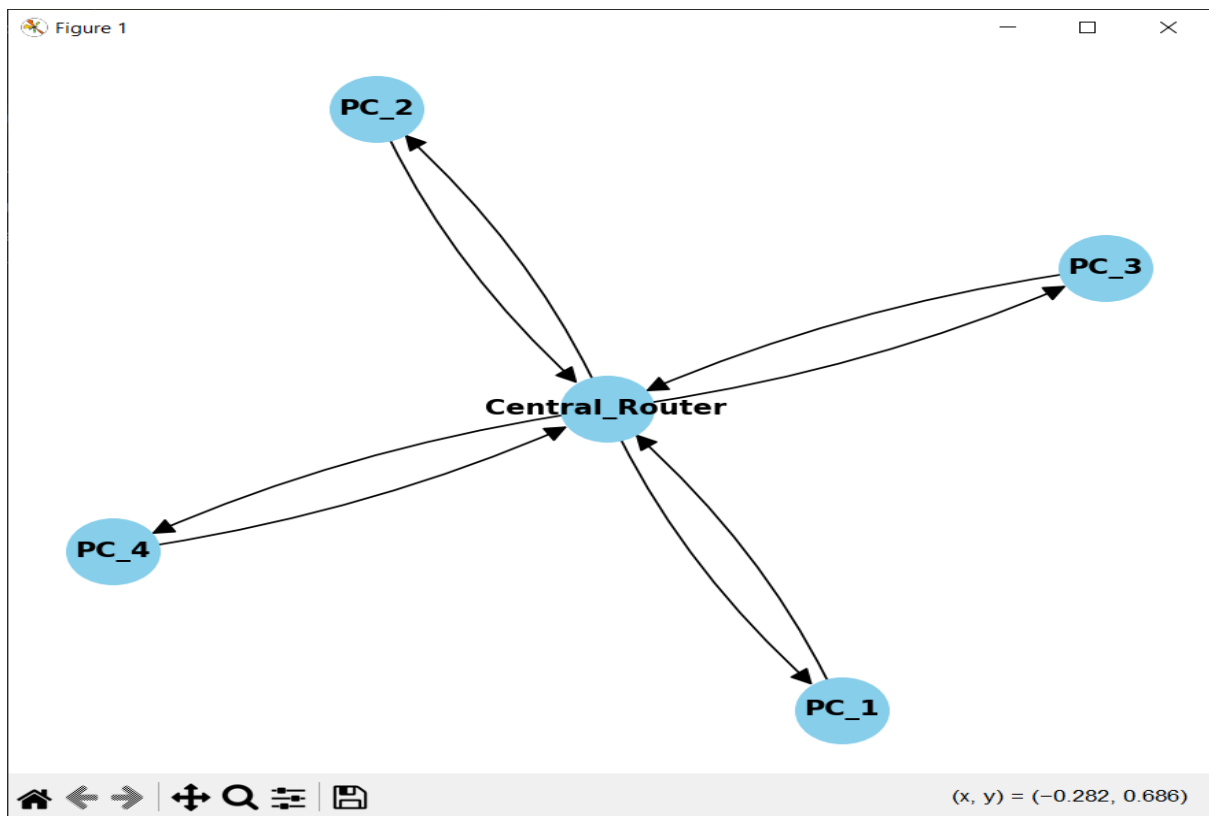


Рис.5(Зіркова топологія)

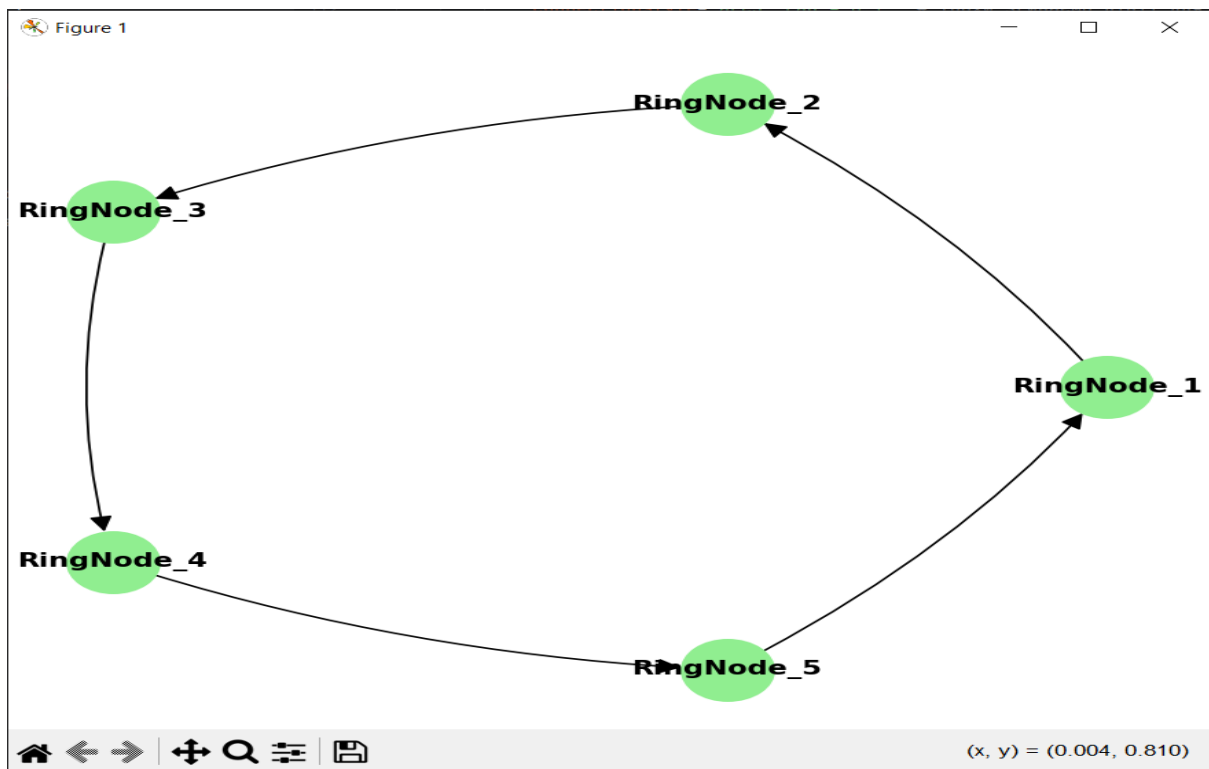


Рис.6(Кільцева топологія)

Висновок

У ході виконання лабораторної роботи було опановано методи моделювання телекомунікаційних мереж з використанням об'єктно-орієнтованого програмування на мові Python. Було реалізовано програмну модель мережі, що включає класи для вузлів, маршрутизаторів та пакетів, а також симульовано передачу даних за протоколами TCP та UDP. Використання бібліотеки `asuncio` дозволило змодельовати асинхронну природу мережевого трафіку з урахуванням випадкових затримок та втрат пакетів. Завдяки бібліотеці `networkx` було візуалізовано обрані топології та проаналізовано їхню ефективність на основі отриманих метрик продуктивності.